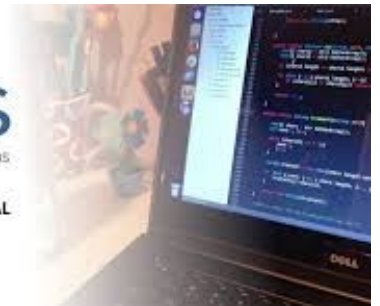


Tecnologia em Análise e Desenvolvimento de Sistemas – TADS

Estrutura de Dados

Prof. Luciano Vargas Gonçalves
E-mail: luciano.goncalves@riogrande.ifrs.edu.br



Estrutura da Dados

Aula 4 – Estruturas Dados Dinâmicas

- **Listas**

Estrutura da Dados

Listas Lineares – Tipos:

- **Simplemente Encadeada – LSE**
- **Duplamente Encadeada - LDE**

Lista Linear

- **LISTA LINEAR :**

- É uma coleção de **elementos** (Nós) do mesmo tipo, dispostos linearmente, que podem ou não seguir determinada organização.
 - Por exemplo: lista de telefone, lista carros, lista filmes, etc;
- A lista dinâmica é composta por **um número variado de elementos (n) / limitado pela memória RAM disponível** (E0,E1,E2,...En);
 - Onde “n” é a quantidade de elementos ($n \geq 0$);
- O fim da lista é marcado pela terminador (“**NULL**”);
- O **ponteiro próximo** e/ou **anterior** determinam o encadeamento dos elementos.



Lista Simplesmente Encadeada de Elementos

Estrutura de dados - Listas

- **Tipos de Listas Encadeadas:**

- **Simples** (Deslocamento em apenas *um sentido* – *Um ponteiro = “Próximo”*)



- **Duplamente** (Deslocamento em *ambos sentidos* – *dois ponteiros “Anterior e Próximo”*)

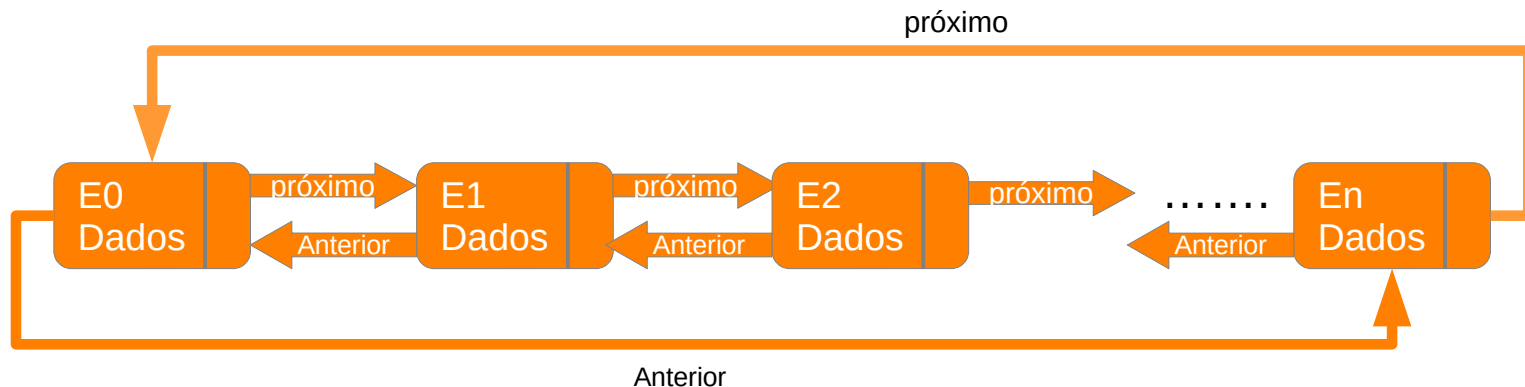


Estrutura de dados - Listas

- **Tipos de Listas Simplesmente Encadeadas:**

- **Circulares**

- Ponteiros para Anterior e Próximo não apontam para NULL nos extremos, apontam para o primeiro e último.
 - Não possuem terminadores (NULL)



Lista Encadeada / VETOR

	VETOR	LISTA ENCADEADA
Tamanho	FIXO	Variável
Alocação	ESTÁTICA	Dinâmica
Acesso	DIRETO	INDIRETO
Uso de Índex	I, j	Ponteiros

Estrutura de dados - Listas

- Quando falamos de listas dinâmicas:
 - Precisamos pensar nos **elementos da estrutura (informações)**:
 - Ex:
 - Lista de Chamada: os elementos são os alunos matriculados;
 - Lista de Carros: os elementos são os carros;
 - Precisamos pensar **na interface da lista (conjunto de funções (API))**:
 - Ex:
 - Funções de Inserção / Remoção, Consulta e Alteração de elementos.

Elemento de Lista - Nó

- **Elemento da Lista ou Nó da lista**

- É uma estrutura de dados que irá armazenar:

- **DADOS:** As informações do elemento da lista (dados);
 - **PONTEIRO(S):** Anterior e/ou Próximo da lista;
 - Exemplo:
 - A Lista de chamada armazena alunos;
 - A Lista de Filmes armazena títulos de filmes;
 - A Lista de tarefas armazena as atividades pendentes;



Elemento de Lista

- Exemplo: Lista de Chamada de Alunos
 - O elemento será Aluno;
 - Onde iremos armazenar as informações de um Aluno:
 - Nome, idade, matricula;

Aluno
Nome Idade Matrícula

Elemento Aluno

Elemento de Lista

- Elementos na memória RAM;
 - Os elementos ficam dispersos na memória quando **alocados dinamicamente**;
 - Cada elemento é alocado no espaço disponível no momento da alocação;
 - Exemplo:
 - Alunos: João, Maria e Paula

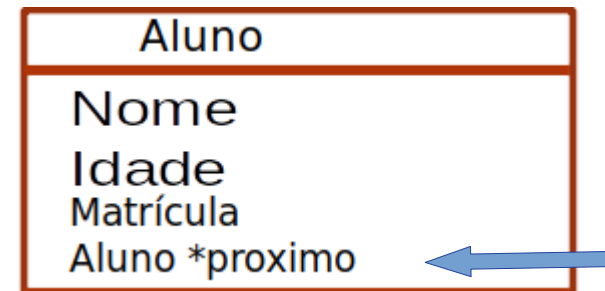
Endereço	Dados
xxx01	Aluno João
xxx02	
xxx03	
.....	
xxx50	
xxx51	Aluna Maria
.....	
.....	...
xxx81	Aluna Paula
xxx82	

Ex: Memória RAM

Listas Simplesmente Encadeada - LSE

- **LSE - Ponteiro próximo:**

- Para encadear (ligar) os elementos de modo que cada elemento aponte para o próximo elemento, iremos criar um apontador que irá armazenar uma referência (ponteiro) para o “**próximo**” elemento da lista (LSE).
- Por exemplo, no caso de uma lista de Alunos (*próximo Aluno).
 - **Próximo** é um ponteiro de encadeamento (ligação) para formação da lista.



```
typedef struct Aluno{  
    char nome [20];  
    int idade;  
    int matricula;  
    struct Aluno *proximo;  
}Aluno;
```

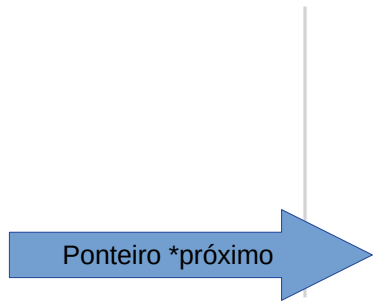
A blue arrow points from the right side of the 'proximo' field in the code to the right, indicating it points to the next element in the list.

Listas Simplesmente Encadeada - LSE

- **Estrutura de Dados para armazenar Alunos**

- Elemento **Aluno** armazena os dados de um aluno e o ponteiro para o próximo **Aluno** (**proximo*);

```
typedef struct Aluno{  
    char nome [20];  
    int idade;  
    int matricula;  
    struct Aluno *proximo;  
}Aluno;
```



A blue arrow points from the text 'Ponteiro *proximo' to the 'proximo' field in the struct definition. The arrow is labeled 'Ponteiro *proximo' and points to the 'proximo' field in the struct definition.

Exemplo de estrutura para armazenar dados de um Aluno

Elemento de Lista

- Elementos na memória RAM;
 - Os elementos ficam dispersos na memória quando **alocados dinamicamente**;
 - Ponteiro próximo que faz o encadeamento (ordem) dos elementos



Exemplo de Lista Simplesmente Encadeada - LSE

Endereço	Dados
xxx01	João *próximo &Paula
....	
xxx50	
xxx51	Maria *próximo &NULL
.....	
.....	...
xxx81	Paula *próximo &Maria
xxx82	

Ex: Memória RAM

LSE - Gerenciamento

- Estrutura para gerenciar a Lista
 - Pode ser um estrutura com ponteiros para início e/ou fim;

```
typedef struct LSE{  
    Aluno *primeiro;  
    int n;  
}LSE;
```

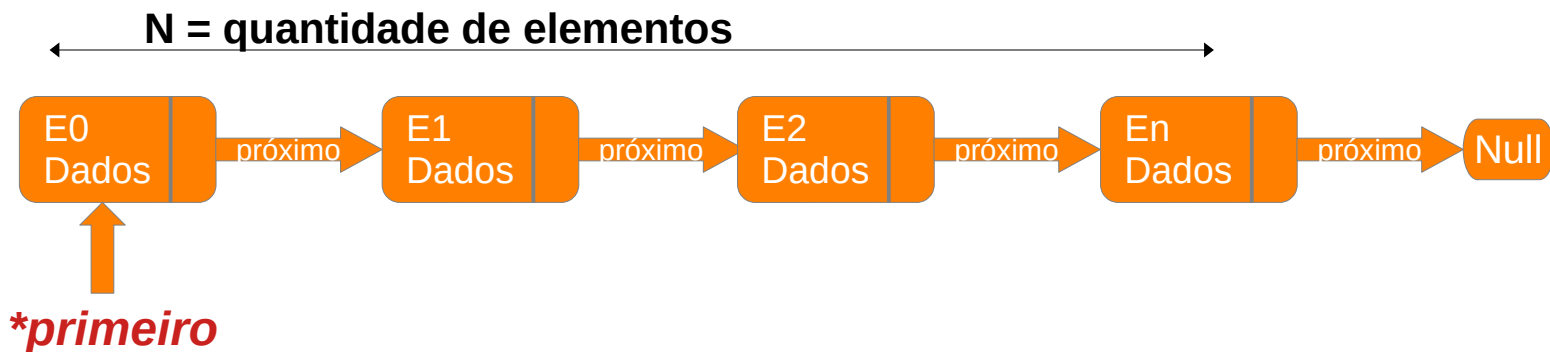
```
// declaração da Lista  
LSE matematica;  
// inicialização da Lista  
inicializaLista(&matematica, "Matematica");
```

- Pode ser apenas um ponteiro para início da lista – Lista Sem cabeça

```
// declaração da Lista  
Aluno *matematica = NULL;  
// inicialização da Lista
```

Listas Simplesmente Encadeada - LSE

- **Estrutura para gerenciar Listas Simplesmente Encadeada – LSE**
 - Tamanho da lista (*n° de elementos*)
 - Lista pode ter de 0 ou mais elementos encadeados (n);
 - *Pontos de acesso e a navegação :*
 - **Ponteiro *Primeiro** marca o início da Lista (Obrigatório);
 - *Ponteiro *Último* marca o fim da lista (facultativo)



Estrutura de Dados para LSE

- **Definição da Estrutura da LSE**
 - Estruturas de dados (LSE) para gerenciar uma Lista Simplesmente Encadeada.
 - **Exemplo 1, apenas um ponteiro para o início da lista:**

```
typedef struct LSE{  
    Aluno *primeiro;  
    int n;  
}LSE;
```

Início da lista (ponteiro *primeiro)

Quantidade de elementos

Estrutura de Dados para LSE

```
// Estrutura para armazenar alunos
typedef struct aluno
{
    char nome[100];
    int idade;
    int matricula;
    struct aluno *proximo;
} Aluno;

// Estrutura para gerenciar a lista
typedef struct lse
{
    Aluno *primeiro;
    int n_elementos;
    char nome[50];
} LSE;
```

```
int main()
{
    printf("Lista Simplesmente Encadeada - Lista de Alunos\n");
    // declaração da Lista
    LSE matematica;

    // Ponteiro de lista e alocação dinâmica
    LSE *portugues;
    portugues = (LSE *)malloc(sizeof(LSE));

    // declara e inicializa um novo elemento da lista de alunos
    Aluno joao = {"Joao Pedro", 23, 12345};
    joao.proximo = NULL;
}
```

Programa principal - main()

Definição de Aluno e Lista

Criar uma nova Lista(LSE)

- **Declarar uma nova lista como Estrutura**

- Uma nova lista LSE para turma de matemática.

```
//declaração da Lista
```

```
LSE matematica;
```

- **Lista de Português alocada dinamicamente**

```
//declarar um ponteiro de lista e alocar memória
```

```
LSE *portugues = (LSE *) malloc(sizeof(LSE));
```

Função para uma LSE

- InicializaLista ()
 - Inicializa os parâmetros de nova Lista alocada dinamicamente
 - Recebe um ponteiro de Lista (*listas
 - Inicializa o ponteiro ***primeiro** e **quantidade** de elementos para lista vazia.

```
void inicializaLista(LSE *lista)
{
    lista->primeiro = NULL;
    lista->n_elementos = 0;
}
```

Função para uma LSE

- `criaLista ( )`
 - Aloca memória para uma lista;
 - Inicializa os parâmetros da nova Lista alocada dinamicamente;
 - Recebe um ponteiro de ponteiro de Lista (`**lista`)
 - Inicializa o ponteiro ***proximo** da lista.

```
void criaLista(Aluno **lista, char nome[])  
{  
    *lista = (Aluno*)malloc(sizeof(Aluno));  
    (*lista)->proximo = NULL;  
}
```

Programa Principal – Main ()

- Novo Elemento Aluno de lista, “Joao”
 - Declara e inicializa um novo aluno;

```
typedef struct aluno
{
    char nome[100];
    int idade;
    int matricula;
    struct aluno *proximo;
} Aluno;
```

```
//declara e inicializa um novo elemento de lista de alunos
Aluno joao = {"Joao Pedro", 23, 12345};
joao.proximo = NULL;
```

Função para mostrar um Aluno

- Função Mostra Aluno (passado por valor)
 - Recebe um aluno e mostra os dados do aluno;

```
void mostraAluno(Aluno aluno){  
    printf("\n Dados do Aluno:");  
    printf("\n\t Nome: %s",aluno.nome);  
    printf("\n\t Idade: %d",aluno.idade);  
    printf("\n\t Matricula: %d\n",aluno.matricula);  
}
```

Programa Principal – Main ()

- Função Mostra Aluno

```
void mostraAluno(Aluno aluno){  
    printf("\n Dados do Aluno:");  
    printf("\n\t Nome: %s",aluno.nome);  
    printf("\n\t Idade: %d",aluno.idade);  
    printf("\n\t Matricula: %d\n",aluno.matricula);  
}
```

Nome:Pedro

Idade:44

Matricula:1123301

Saída no terminal

Cadastra dados Aluno

- **Função cadastraAluno()**

- Recebe um ponteiro Aluno e os dados do novo aluno (nome, idade, matricula)

```
void cadastraAluno (Aluno *aluno, char nome[], int idade, int matricula)
{
    strcpy(aluno->nome, nome);
    aluno->idade = idade;
    aluno->matricula = matricula;
    aluno->proximo = NULL;
}
```

Cadastra dados Aluno

- **Função cadastraAluno()**

- Alocar um novo aluno e preencher os dados

```
//novo aluno declarado  
Aluno paulo;
```

```
//casdatrar um novo aluno por meio de uma função  
cadastraAluno(&paulo,"Paulo Roberto",34,232323);
```

```
void cadastraAluno (Aluno *aluno, char nome[],int idade, int matricula)  
{  
    strcpy(aluno->nome, nome);  
    aluno->idade = idade;  
    aluno->matricula = matricula;  
    aluno->proximo = NULL;  
}
```

Exemplo2.c

- Cadastro de 3 alunos
 - João, Paulo e Maria
 - Alocação Estática

```
// programa principal para teste
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "LSE.h"
int main()
{
    printf("Lista Simplesmente Encadeada - Lista de Alunos\n");
    // declaração de ponteiro de Lista
    LSE *matematica;
    // inicialização da Lista
    criaLista(&matematica, "Matematica");

    // declara e inicializa um novo elemento da lista de alunos
    Aluno joao = {"Joao Pedro", 23, 12345};
    joao.proximo = NULL;

    // novo aluno declarado estaticamente
    Aluno paulo;
    cadastraAluno(&paulo, "Paulo Roberto", 34, 232323);

    Aluno maria;
    cadastraAluno(&maria, "Maria", 21, 334343);

    insereInicio(matematica, &joao);
    insereInicio(matematica, &paulo);
    insereInicio(matematica, &maria);
    mostraLista(*matematica);

    exit(0);
}
```

Interface da LSE

- A **interface da lista** implementa as funções de Lista:
 - **Funções de Inserção:**
 - InsereNoInicio(), InsereNoFim(), InsereNaPosição()
 - **Funções de Remoção:**
 - RemoveNoInicio(), RemoveNoFim(), RemoveNaPosição();
 - **Funções de Consulta:**
 - MostraElemento(), MostraLista()
 - **Funções de Exclusão:**
 - ApagaElemento(), ApagaLista()
- **OBS:**

A Lista LSE não possui restrição de inserção e/ou remoção.

Funções de Inserção

- **Como Inserir um novo elemento na Lista, Funções:**
 - **InserNoInicio():**
 - Insere um novo elemento na primeira *posição (E0)*;
 - **InserNoFim()**
 - Insere um novo elemento na última *posição (En)*;
 - **InserNaPosição (posicao)**
 - Insere um novo elemento na *posição determinada (posição)*;
- **ATENÇÃO:** Condições a serem avaliadas na inserção de elementos:
 - Lista vazia ($n_elementos = 0$);
 - Lista com elementos ($n_elementos > 0$);

Interface da LSE – Insere no Início

- Método para inserir um **novο** elemento no **Início** na Lista:
 - Duas situações precisam ser avaliadas;

1) Lista vazia;

- Ponteiro (***primeiro == NULL**) ou
- Número elementos (**n == 0**)



2) Lista com elementos;

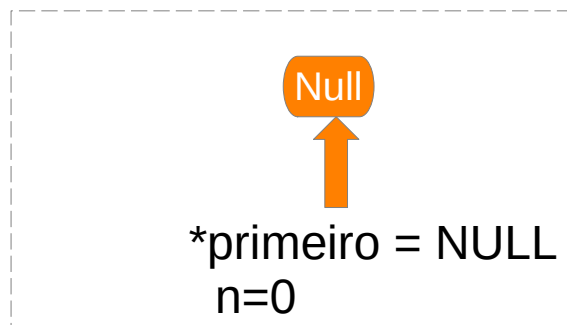
- Ponteiro (***primeiro != NULL**) ou
- Número de elementos (**n > 0**);



Inserir no Início Lista Vazia

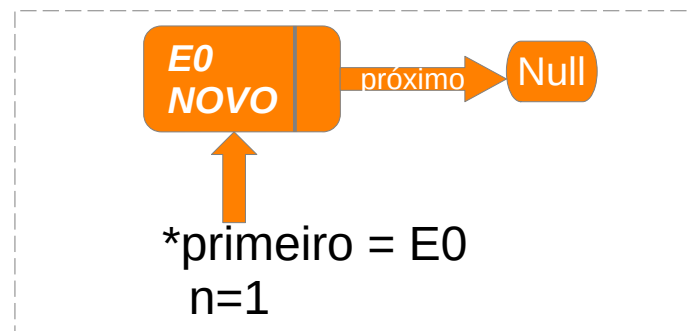
- Método para inserir um **novο** elemento no **Início** na Lista

Estado Atual – Lista Vazia



Antes da Inserção

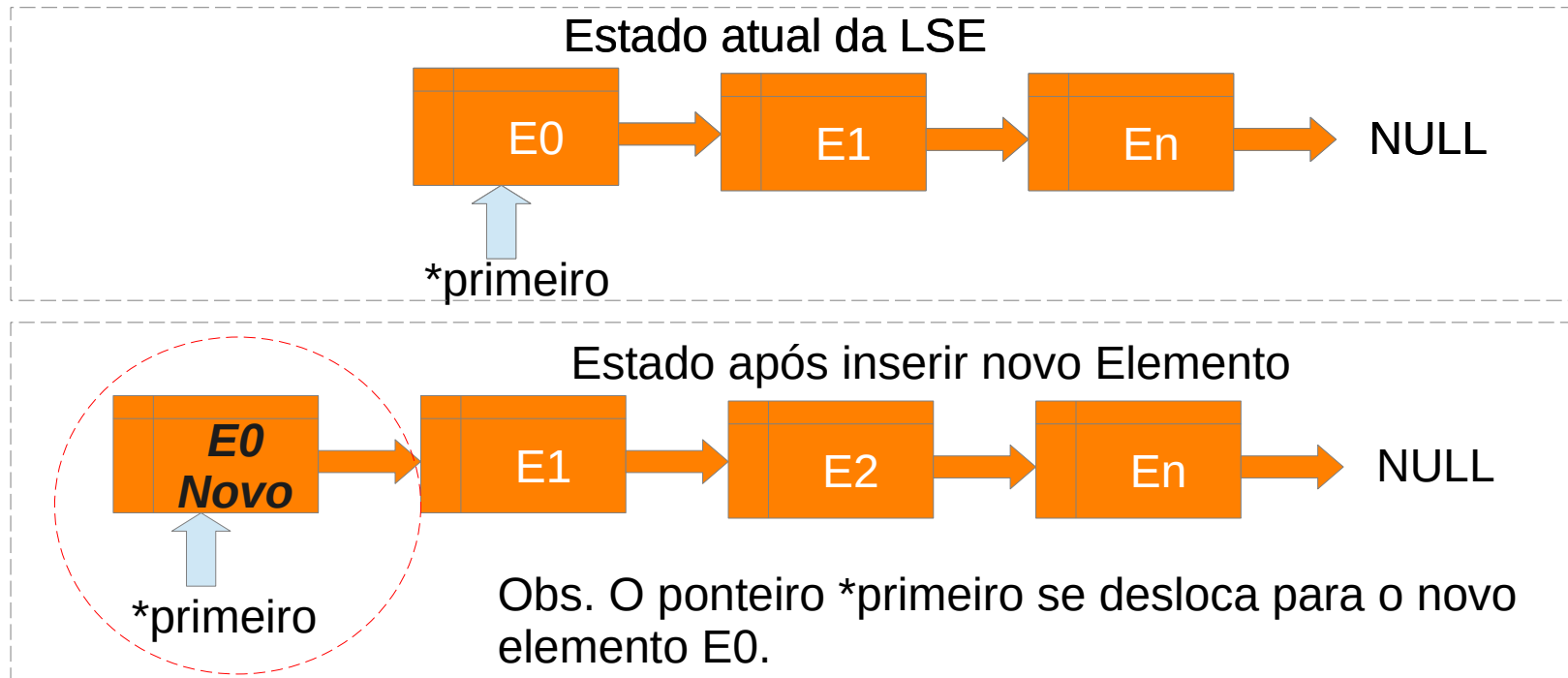
Após Inserção



Após inserção

Inserir no Início da Lista com elementos

- Método para inserir um **novο** elemento no **Início** na Lista
 - Estado Atual – Lista COM ELEMENTOS ($N > 0$);



Interface da LSE – Insere no Início

- Método para inserir um **novο** elemento no **Início** na Lista:
 - A função recebe um ponteiro de lista (***ls**) e um ponteiro de elemento (***aluno**)
 - Situações a serem avaliadas;

1) Lista vazia;

- Ponteiro (*primeiro == NULL);
- Número elementos (n == 0)

2) Lista com elementos;

- Ponteiro (*primeiro != NULL);
- Número de elementos (n > 0);

```
void insereInicio(LSE *lista, Aluno *novo){  
    if(lista->primeiro == NULL){  
        //Lista Vazia  
        novo->proximo = NULL;  
    }else{  
        //Lista com elementos  
        novo->proximo = lista->primeiro;  
    }  
    lista->primeiro = novo;  
    lista->n_elementos++;  
}
```

- A função recebe dois ponteiros, um para lista (*ls) e um para novo elemento (*novo);

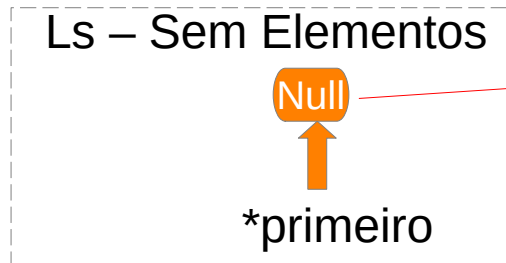
Interface da LSE – Insere no Início

- Método para inserir um *novο* elemento no *Início* na Lista:
 - A função insere no início reduzida..
 - A validação para lista vazia na função insere no início é facultativa.

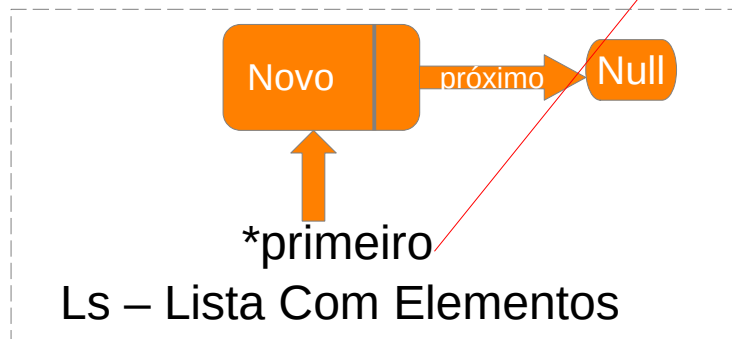
```
void insereInicioReduzida(LSE *lista, Aluno *novo){  
    novo->proximo = lista->primeiro;  
    lista->primeiro = novo;  
    lista->n_elementos++;  
}
```

Interface da LSE – Insere no Início

- Método para inserir um **novο** elemento no **Início** na Lista
 - Estado Atual – Lista Vazia



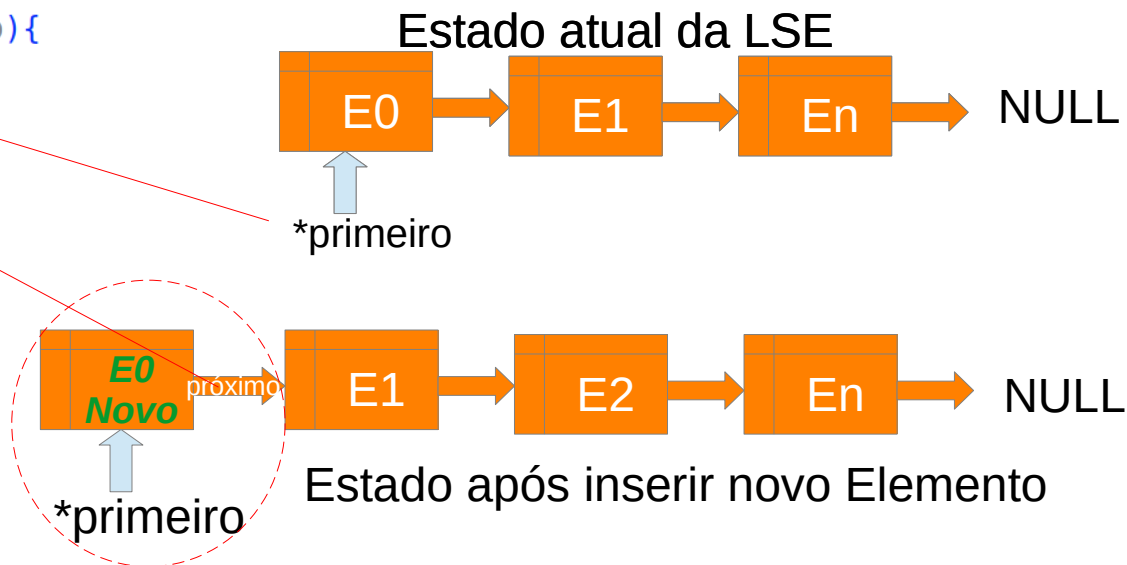
```
void insereInicioReduzida(LSE *lista, Aluno *novo){  
    novo->proximo = lista->primeiro;  
    lista->primeiro = novo;  
    lista->n_elementos++;  
}
```



Interface da LSE – Insere no Início

- Método para inserir um **novο** elemento no **Início** na Lista
 - Estado Atual – Lista COM ELEMENTOS ($n > 0$)

```
void insereInicioReduzida(LSE *lista, Aluno *novo){  
    novo->proximo = lista->primeiro;  
    lista->primeiro = novo;  
    lista->n_elementos++;  
}
```



Interface da LSE – Insere no Início

- Programa principal inserir novo elemento na LSE;

```
//função para inserir um novo aluno na lista de matemática
```

```
insereInicio(&matematica,&joao);
```

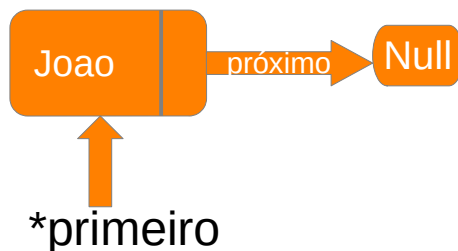
```
insereInicio(&matematica,&paulo);
```

```
void insereInicio(LSE *lista, Aluno *novo){  
    if(lista->primeiro == NULL){  
        //Lista Vazia  
        novo->proximo = NULL;  
    }else{  
        //Lista com elementos  
        novo->proximo = lista->primeiro;  
    }  
    lista->primeiro = novo;  
    lista->n_elementos++;  
}
```

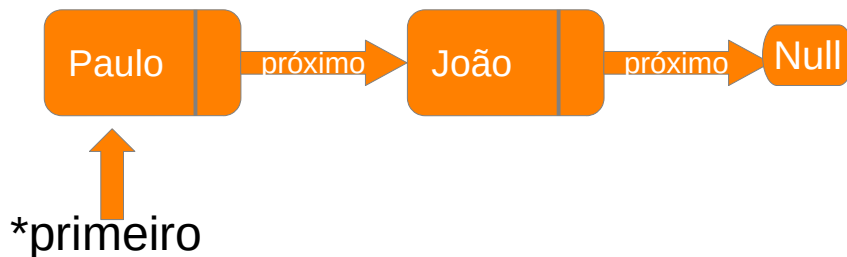
Interface da LSE – Insere no Início

- Método para inserir um **novο** elemento no **Início** na Lista

Estado atual da LSE



Estado atual da LSE – Após inserção



OBS: Ordem de Inserção é mantida!!!

Interface da LSE – Mostra Lista

- **MostraLista()**

- Mostra todos elementos a partir do primeiro elemento;
- Necessário para varrer a lista (iterar);
- Ponteiro auxiliar *aux;

```
void mostraLista(LSE lista){  
    printf("\n Mostra Lista Simplesmente Encadeada; \n");  
    Aluno *aux; //ponteiro auxiliar  
    aux = lista.primeiro;  
    for(int i=0;i<lista.n_elementos;i++){  
        mostraAluno(*aux);  
        aux = aux->proximo;  
    }  
    printf("\n Fim da Lista!\n");  
}
```

Interface da LSE – Mostra Lista - FOR

- **MostraLista()**

```
void mostraLista(LSE lista){  
    printf("\n Mostra Lista Simplesmente Encadeada; \n");  
    Aluno *aux; //ponteiro auxiliar  
    aux = lista.primeiro;  
    for(int i=0;i<lista.n_elementos;i++){  
        mostraAluno(*aux);  
        aux = aux->proximo;  
    }  
    printf("\n Fim da Lista!\n");  
}
```

Mostra Lista Simplesmente Encadeada;

Aluno Nome: Maria Idade 21 Matricula 334343

Aluno Nome: Paulo Roberto Idade 34 Matricula 232323

Aluno Nome: Joao Pedro Idade 23 Matricula 12345

Fim da Lista!

Interface da LSE – Mostra Lista - While

- **MostraLista()** - iterativo, sem contador

```
void mostraLista(LSE lista)
{
    // mostra todos elementos da lista
    printf("\n\tMostra Lista Simplesmente Encadeada; \n");
    Aluno *aux; // ponteiro auxiliar
    aux = lista.primeiro;
    while (aux != NULL)
    {
        mostraAluno(*aux);
        aux = aux->proximo;
    }
    printf("\t\tTotal de alunos na Lista de %s = %d \n", lista.nome, lista.n_elementos);
    printf("\t\tFim da Lista!\n");
}
```

Mostra Lista Simplesmente Encadeada;

Aluno Nome: Maria Idade 21 Matricula 334343

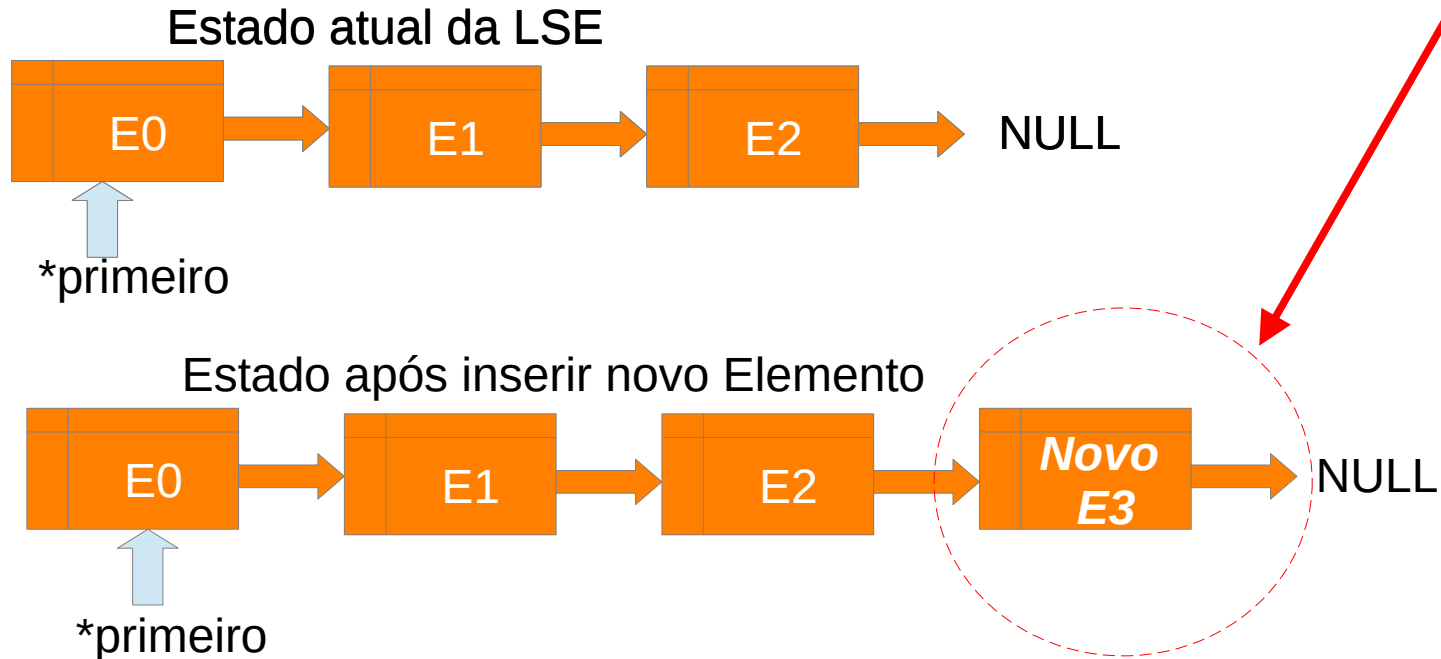
Aluno Nome: Paulo Roberto Idade 34 Matricula 232323

Aluno Nome: Joao Pedro Idade 23 Matricula 12345

Fim da Lista!

Interface da LSE – Insere no FIM

- Método para inserir um **novο** elemento no **Final** na Lista

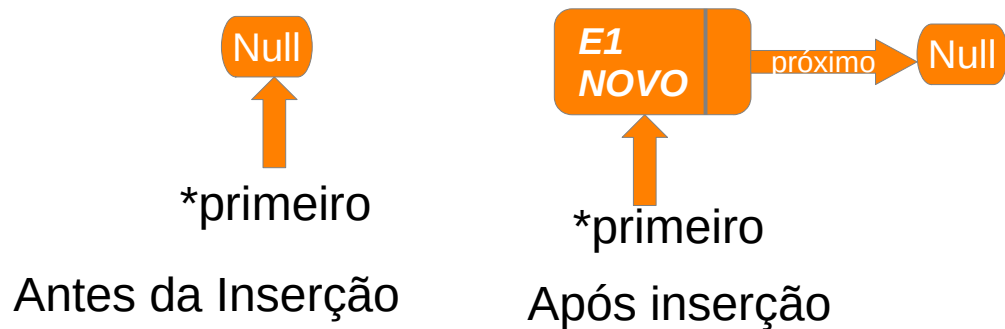


Interface da LSE – Insere no FIM

- **Inserir um elemento no *FIM* da Lista**
 - Função InserirNoFim()
 - Recebe um ponteiro de lista **lista* e um ponteiro de Aluno **novo*;
 - Função sem retorno;
 - Duas possibilidades para inserção :
 - 1) Lista Vazia antes da inserção (*n = 0*);
 - 2) Lista com elementos (*n > 0*);

Inserir no FIM - Lista Vazia

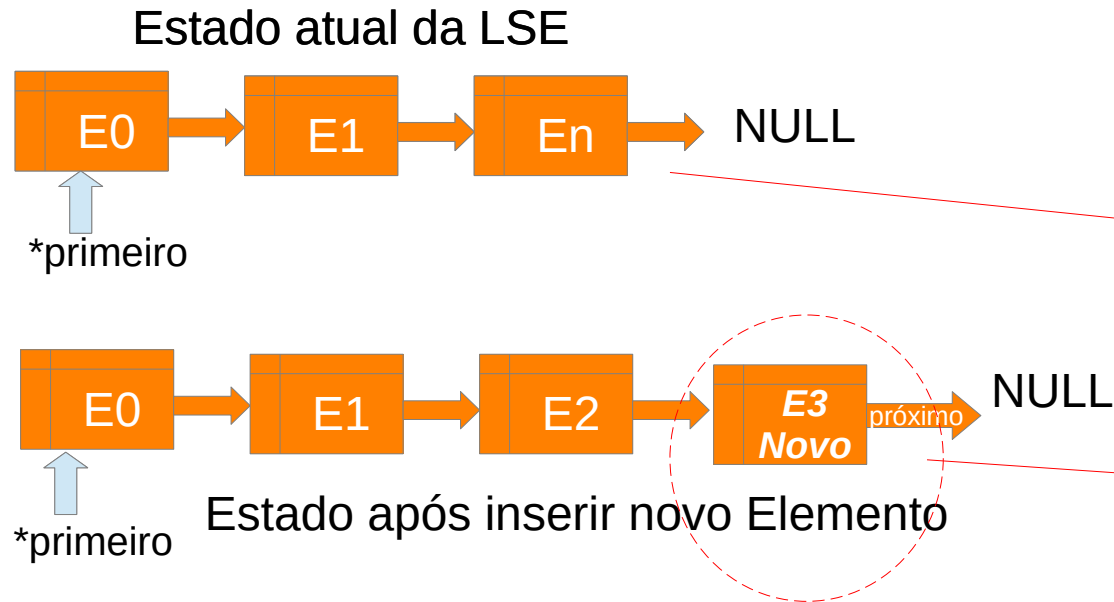
- Método para inserir um **novο** elemento no **FIM** na Lista
 - Estado Atual – Lista Vazia
 - Procedimento idêntico ao Inserir no Início



```
void insereFim(LSE *lista, Aluno *novo){
    novo->proximo = NULL;
    if(lista->primeiro == NULL){
        //Lista Vazia
        lista->primeiro = novo;
    }else{
        //Lista com elementos
    }
}
```

Inserir no FIM - Lista com Elementos

- Método para inserir um **novο** elemento no **FIM** na Lista
 - Estado Atual – Lista COM ELEMENTOS



```
void insereFim(LSE *lista, Aluno *novo){
    novo->proximo = NULL;
    if(lista->primeiro == NULL){
        //Lista Vazia
        lista->primeiro = novo;
    }else{
        //Lista com elementos
        Aluno *aux = lista->primeiro;
        while(aux->proximo!=NULL){
            aux = aux->proximo;
        }
        aux->proximo = novo;
    }
    lista->n_elementos++;
}
```

Funções de Remoção

- **Como remover um novo elemento na Lista, Funções:**

- **Elemento * RemoçãoNoInicio(Lista *):**
 - Remove um elemento no início da lista;
- **Elemento * RemoçãoNoFim(Lista *):**
 - Remove um elemento no fim da lista;
- **Elemento * RemoçãoNaPosicao(Lista ls*, int pos):**
 - Remove um elemento em uma posição específica (pos);
- **Condições a serem avaliadas:**
 - Lista vazia ($n = 0$);
 - Lista com elementos ($n > 0$);

- **Atenção:**

Remover é diferente de excluir um elemento, remover retorna o elemento removido.

Interface da LSE – Remove Início

- **RemoveNoInicio ()**
 - Recebe ponteiro da Lista e Remove o primeiro elemento da Lista
 - Por fim, retorna o endereço do elemento removido;

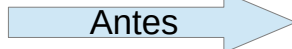
```
Aluno* removeInicio(LSE *lista){  
    Aluno *aux = lista->primeiro;  
    if(aux != NULL){  
        //lista com um ou mais elementos//  
        lista->primeiro = aux->proximo;  
        lista->n_elementos--;  
    }  
    return aux;  
}
```

Interface da LSE – Remove Início

- **RemoveNoInicio ()**

```
Aluno* removeInicio(LSE *lista){  
    Aluno *aux = lista->primeiro;  
    if(aux != NULL){  
        //lista com um ou mais elementos//  
        lista->primeiro = aux->proximo;  
        lista->n_elementos--;  
    }  
    return aux;  
}
```

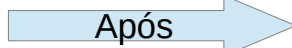
Mostra Lista Simplesmente Encadeada;

 Antes

Aluno Nome: Maria Idade 21 Matricula 334343
Aluno Nome: Paulo Roberto Idade 34 Matricula 232323
Aluno Nome: Joao Pedro Idade 23 Matricula 12345
Aluno Nome: Juca Silva Idade 25 Matricula 3232323
Total de alunos na Lista = 4
Fim da Lista!

Elemento removido foi:
Aluno Nome: Maria Idade 21 Matricula 334343

Mostra Lista Simplesmente Encadeada;

 Após

Aluno Nome: Paulo Roberto Idade 34 Matricula 232323
Aluno Nome: Joao Pedro Idade 23 Matricula 12345
Aluno Nome: Juca Silva Idade 25 Matricula 3232323
Total de alunos na Lista = 3
Fim da Lista!

Tarefa

- Implementar as Funções da Lista LSE:
 - RemoveNoFim():
 - Recebe um ponteiro de lista e remove o elemento na última posição;
 - Retorna um ponteiro *Aluno
 - RemoveNaPosição():
 - Recebe um ponteiro de lista, uma posição e remove o elemento na posição;
 - Retorna um ponteiro *Aluno
 - InsereNaPosição(int posicao):
 - Recebe um ponteiro de lista, uma posição(pos) e um ponteiro de Aluno;
 - Função sem retorno.

Tarefa

- LSE.H

Protótipo das funções
de LSE;

```
void insereNoInicio(LSE *ls, Aluno *novo);  
void insereNoFim(LSE *ls, Aluno *novo);  
void insereNaPosicao(LSE *ls, Aluno *novo, int pos);
```

```
Aluno* removeNoInicio(LSE *ls);  
Aluno* removeNoFim(LSE *ls);  
Aluno* removeNaPosicao(LSE *ls, int pos);
```

```
int retornaQuantidade(LSE *ls);
```

```
void mostraLista(LSE *ls);  
void mostraAluno(Aluno *novo);  
void mostraPosicao(LSE *ls, int pos);
```

```
void apagaLista(LSE *ls);  
void apagaAluno(Aluno *novo);
```

Tarefa

- Função Menu
 - Controla a execução da Lista

```
int menu(LSE *ls){
    printf("1 - Inserir No Início:\n");
    printf("2 - Inserir No Fim:\n");
    printf("3 - Inserir No Posição:\n");
    printf("4 - Remove No Início:\n");
    printf("5 - Remove No Fim:\n");
    printf("6 - Remove No Posição:\n");
    printf("7 - Mostra Lista:\n");
    printf("8 - InsereOrdenado \n");
    printf("0 - Sair Programa:\n");
    int op,pos;
    scanf("%d",&op);
    DadosAluno *novo;
    if(op==1){
        novo = lerDadosAluno();
        insereNoInicio(ls,novo);
    }else if(op == 2){
        novo = lerDadosAluno();
        insereNoFim(ls,novo);
    }else if(op == 3){
        printf("\nInforme a Posição:");
        scanf("%d",&pos);
        novo = lerDadosAluno();
        insereNaPosicao(ls,novo,pos);
    }else if(op == 4){
        novo = removeNoInicio(ls);
    }else if(op == 5){
        novo = removeNoFim(ls);
    }else if(op == 6){
        printf("\nInforme a Posição:");
        scanf("%d",&pos);
        removeNaPosicao(ls,pos);
    }else if(op == 7){
        mostraLista(ls);
    }else{
        printf("\nFim da Execução!\n");
    }
    return op;
}
```

Tarefa

- Execução da Lista

```
luciano@luciano-pc:~/Documentos/IFRS-2021/ED/Code-C/Aula4/Referencia$ ./roda
```

Nome:Pedro	Idade:34	Matricula:1123301
Nome:Paulo	Idade:44	Matricula:1123301
Nome:Maria	Idade:54	Matricula:1123301
Nome:Paula	Idade:24	Matricula:1123301

Fim da Lista

Remove no Início:

Nome:Paulo	Idade:44	Matricula:1123301
Nome:Maria	Idade:54	Matricula:1123301
Nome:Paula	Idade:24	Matricula:1123301

Fim da Lista

- 1 - Inserir No Início:
- 2 - Inserir No Fim:
- 3 - Inserir No Posição:
- 4 - Remove No Início:
- 5 - Remove No Fim:
- 6 - Remove No Posição:
- 7 - Mostra Lista:
- 0 - Mostra Lista:

Exercícios

- **Implemente as demais funções de *lista*:**
 - **Funções de Inserção:**
 - `InserNaPosição(LSE *lista, int pos)`
 - Insere um elemento na lista na posição “pos”
 - **Funções de Remoção:**
 - `RemoveNoFim(LSE *lista)`
 - Remove o elemento que está no final da lista;
 - `RemoveNaPosição(LSE *lista, int pos);`
 - Remove o elemento que está na posição “pos”;
 - **Funções de Exclusão:**
 - `ApagaElemento(Aluno *aluno),`
 - Apaga um elemento recebido “Free”
 - `ApagaLista(LSE *lista)`
 - Apaga todos elementos da Lista;

Exercícios

- Crie uma turma com 10 alunos de matemática. Mostre todos alunos;
- Copie os alunos da turma de matemática para turma de português. Mostre os alunos das duas turmas;
- Mostre a turma de português em ordem invertida;
- Insira dois novos alunos na turma de matemática nas posições 5 e 15. Mostre a turma.

Material de Apoio

- IFRS – Pergamum
 - SZWARCFITER, Jayme Luiz. Estruturas de dados e seus algoritmos. 3. Rio de Janeiro LTC 2010 1 recurso online ISBN 978-85-216-2995-5.
 - DEITEL, Paul; Deitel, Harvey. C: como programar - 6ª edição. Editora Pearson 850 ISBN 9788576059349.